

Humans versus AI: A Multi-Dimensional Characterization of Code Quality and Security

TODO Author  

TODO Affiliation, Country

Abstract

Background. TODO: Context and motivation.

Aims. TODO: Research goals and questions.

Method. TODO: Dataset scale, tooling, statistical methods, and analysis pipeline.

Results. TODO: Main empirical findings with quantitative evidence.

Conclusions. TODO: Implications and broader interpretation.

2012 ACM Subject Classification Software and its engineering → Software defect analysis; Software and its engineering → Software testing and debugging

Keywords and phrases TODO: comma-separated keywords

Digital Object Identifier 10.4230/LIPICs.TODO.2025.TODO

Funding *TODO Author:* TODO funding

Acknowledgements TODO: Acknowledgments text. Include funding sources, data providers, or reviewers as appropriate.

1 Introduction

AI-assisted programming tools have become a standard part of modern software development. Systems such as GitHub Copilot, Devin, and Cursor now automate tasks ranging from code completion to pull request creation, operating with minimal human oversight. The scale of this adoption is substantial: the AIDev dataset documents over 980,000 AI-generated contributions spanning more than 116,000 repositories [9]. We draw on this dataset to examine how AI authorship affects code quality across a broad range of open-source projects.

While automation accelerates development, it also introduces quality concerns that differ in character from those typically associated with human-authored code. Existing research shows that AI-generated code exhibits higher error rates than human-authored code [3, 17, 8], yet several practical questions remain unanswered: (1) which violation categories and severity levels predominate in AI-generated code, (2) how structural complexity metrics relate to defect density and whether these relationships differ between AI and human-authored code, and (3) which programming languages face the largest quality gaps and whether static type systems reduce AI-associated risks.

We address these questions through a large-scale empirical investigation of 19,539 files drawn from the AIDev dataset, applying three complementary analyses: language-specific linters to assess functional correctness, structural complexity measurements to assess architectural quality, and security scanners to characterize CWE patterns. All analyses are stratified by programming language, enabling direct comparison of AI-generated and human-authored code across multiple quality dimensions. The main contributions of this work are as follows:

- 1. Large-scale empirical assessment of AI-generated versus human-authored code quality.** We analyze 19,539 files across six programming languages using a multi-dimensional framework covering functional, structural, and security quality dimensions, enabling systematic characterization of quality differences across language families and authorship types.



© TODO Author Names;
licensed under Creative Commons License CC-BY 4.0

TODO Conference Name (TODO 2025).

Editors: TODO; Article No. TODO; pp. TODO:1–TODO:7



Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2. **Evidence that type safety reduces AI-associated quality gaps.** Statically-typed languages (Java, C#, TypeScript) exhibit significantly narrower violation density gaps between AI-generated and human-authored code (1.32x to 1.66x) compared to the 2.34x gap observed in dynamically-typed languages. Chi-square analysis confirms this pattern ($p < 0.001$), suggesting that compiler-enforced type checking intercepts a class of errors that otherwise propagate in dynamic languages.
3. **Identification of divergent structural complexity patterns in AI-generated code.** We provide empirical evidence that AI-generated code exhibits an ‘inverted’ quality profile characterized by high cyclomatic complexity alongside low structural coupling, a pattern statistically distinct from human-authored code ($p < 0.05$). This finding indicates that the structural composition of AI-generated code does not conform to assumptions underlying conventional quality metrics.

2 Related Work

2.1 TODO: Subsection Title

TODO: Synthesize prior work thematically. Compare findings across studies and identify unresolved gaps. Separate pre-AI era findings from AI era findings where relevant.

2.2 TODO: Subsection Title

TODO: Position the current work relative to the closest prior studies. Highlight what this paper addresses that prior work does not.

3 Experiment Design

We characterize defect patterns across three analytical dimensions: functional correctness, structural complexity, and security vulnerabilities.

3.0.1 Notes

A statistically significant regression slope ($p < 0.05$) is interpreted as evidence of a trend. In cases where the data may violate linearity or normality assumptions, we apply the nonparametric Mann-Kendall trend test [12], which is robust to non-normal distributions and outliers. This combined methodology ensures the reliability and validity of our assessment of issue prevalence trends across heterogeneous language and authorship combinations.

3.1 RQ1: Functional Correctness and Defect Prevalence

RQ1: *Does AI-generated code exhibit different patterns of linter violations and defect types compared to human-written code across various programming languages?*

Using the linter toolchain described in §4.2, we compute violation rates per file and test statistical significance using chi-square tests (χ^2 , $\alpha = 0.05$) with Cohen’s h effect sizes. We break down violations by severity level and identify top violation messages to characterize predominant issue types. Finally, we stratify by language type, comparing statically typed languages (C, C++, C#, Java, Go) against dynamically typed languages (Python, JavaScript, PHP, Ruby) to test whether static type systems mitigate defect risks in AI-generated code. HTML is analyzed separately as a markup language and excluded from this typed-language comparison.

3.2 RQ2: Structural Complexity and Code Composition

RQ2: *Do structural characteristics such as complexity, coupling, and lexical volume differ statistically between AI-generated and human-written code?*

Traditional software quality models assume structural complexity metrics predict defect density. We test whether this assumption generalizes to AI-generated code using the 27 structural metrics described in §4.2. We compute Pearson correlations between each metric and violation counts separately for AI and human samples, then apply Fisher’s z-transformation to test whether correlation strengths differ significantly between authorship types ($\alpha = 0.05$, Bonferroni-corrected). This analysis tests whether AI code breaks traditional complexity-quality relationships through unexpected patterns such as high control-flow complexity paired with low architectural coupling.

3.3 RQ3: Security Vulnerability Patterns

RQ3: *What security vulnerability patterns are most prevalent in AI-generated code, and how do they vary by language?*

Security vulnerabilities pose direct exploitation risks distinct from general correctness issues. Using the Semgrep analysis protocol described in §4.2, we extract CWE identifiers and severity levels (info, warning, error, critical) for each finding. We compute vulnerability density separately for AI and human code, then apply Mann-Whitney U tests and Cliff’s delta to test whether differences are significant ($\alpha = 0.05$, Bonferroni-corrected). We specifically examine: (1) overall vulnerability density differences between authorship types, (2) which CWE categories dominate AI-generated code, and (3) whether statically typed languages exhibit lower vulnerability rates than dynamically typed languages.

4 Methodology

4.1 Dataset and Data Processing

We use the CodeMirage benchmark [5], a publicly available multilingual dataset hosted on Hugging Face that provides paired human-written and AI-generated source code across ten programming languages: C, C++, C#, Go, HTML, Java, JavaScript, PHP, Python, and Ruby. CodeMirage comprises 10,000 human-written files and approximately 99,993 AI-generated counterparts produced by ten production-level LLMs spanning six providers. Table 1 summarizes the language-level distribution.

The human-written code was drawn from the CodeParrot GitHub-Code-Clean dataset [2], a curated subset of publicly available GitHub repositories sanitized prior to May 2022, before the widespread deployment of code-generating LLMs and autonomous coding agents. This temporal bound ensures that the human baseline is free from AI contamination. For each language, 1,000 files were sampled with additional length-based filtering to preserve diversity while bounding file size to a tractable scale for static analysis.

The AI-generated counterparts were produced using a two-stage text-to-code strategy. Each human-written file was first summarized by an LLM, capturing its purpose, functionality, logic structure, and key identifiers. A separate LLM was then prompted to generate a functionally equivalent implementation from the summary alone, without access to the original source. This approach prevents direct recitation while preserving functional intent. A rule-based inspector enforced that each generated file remained within acceptable bounds of the original’s line count and character length, and required a BLEU score below 0.5 to prevent near-verbatim copying. Samples failing repeated generation attempts were discarded.

The ten LLMs contributing AI-generated code are Claude-3.5-Haiku, GPT-4o-mini, GPT-o3-mini, Gemini-2.0-Flash, Gemini-2.0-Flash-Thinking, Gemini-2.0-Pro, DeepSeek-V3, DeepSeek-R1, Llama-3.3-70B, and Qwen-2.5-Coder-32B [5]. For our quality characterization, we aggregate all AI-generated files across the ten LLMs into a single AI-authored category, allowing us to characterize properties of AI-generated code broadly rather than attributing findings to any specific model.

We restrict our structural complexity analysis (RQ2) to the nine compiled or interpreted programming languages, excluding HTML. Metrics such as cyclomatic complexity and MaxInheritanceTree are defined for programming language constructs and are not meaningful for HTML markup. All other analyses include HTML where applicable tooling is available.

■ **Table 1** Dataset Distribution by Language

Language	AI Files	Human Files	Total
Python	10,000	1,000	11,000
Java	10,000	1,000	11,000
JavaScript	10,000	1,000	11,000
C++	9,999	1,000	10,999
C	10,000	1,000	11,000
C#	9,997	1,000	10,997
Go	10,000	1,000	11,000
PHP	9,998	1,000	10,998
Ruby	10,000	1,000	11,000
HTML	9,999	1,000	10,999
Total	99,993	10,000	109,993

4.2 Static Analysis Toolchain

To characterize qualitative differences between AI-generated and human-written code across multiple dimensions, we employ established static analysis tools to measure functional correctness, structural complexity, and security vulnerabilities.

4.2.1 Functional Defect Detection

For Python and Java, we follow the approach in Liu et al. [10], using Pylint [11] for Python and PMD [16] for Java. For JavaScript, we employ ESLint [21], a widely adopted linter for detecting syntax errors and style violations. For C and C++, we use Cppcheck [13], which captures a broad range of issues including undefined behavior, resource leaks, and portability concerns across both languages. For C#, we use Roslyn [14], integrated into the .NET SDK for syntax, semantic, and maintainability analysis. For Go, we apply staticcheck [6], a comprehensive static analysis tool that identifies correctness bugs, deprecated API usage, and performance issues specific to the Go language. For PHP, we use PHPStan [15], which detects type errors, undefined variables, and logic mistakes without code execution. For Ruby, we apply RuboCop [18], which checks for style violations, potential bugs, and complexity issues. For HTML, we use HTMLHint [7] to detect syntax errors and structural issues in markup files. Table 2 summarizes the linter coverage and violation counts across our dataset.

■ **Table 2** Linter coverage and violation counts per language.

Language	Linters	# Files	# Violations
Python	Pylint	11,000	—
Java	PMD	11,000	—
JavaScript	ESLint	11,000	—
C	Cppcheck	11,000	—
C++	Cppcheck	10,999	—
C#	Roslyn	10,997	—
Go	staticcheck	11,000	—
PHP	PHPStan	10,998	—
Ruby	RuboCop	11,000	—
HTML	HTMLHint	10,999	—

154 **4.2.2 Structural and Complexity Profiling**

155 Existing works on program comprehension reveal that software metrics can be a valuable
156 source of information for understanding the properties of a piece of software [4, 20, 22].
157 We use Understand by SciTools [19] to extract software metrics from the nine compiled
158 and interpreted programming languages in our dataset, excluding HTML as noted in §4.1.
159 Understand is an industry-standard tool for software analytics with support for all target
160 languages. We extract 27 metrics per file capturing three key dimensions of code structure.

161 First, *Volume and Complexity* metrics including CountLineCode, CountStmt, and Cyclo-
162 matic Complexity quantify verbosity and control-flow density. Second, *Lexical Entropy* metrics
163 use Halstead measures (Halstead Effort and Halstead Volume) to assess vocabulary richness
164 and algorithmic complexity. Finally, *Architectural Depth* metrics such as CountClassCoupled
165 and MaxInheritanceTree capture object-oriented coupling and inheritance patterns, applied
166 to object-oriented languages where these constructs are defined (C++, C#, Java, PHP,
167 Python, Ruby). These metrics enable analysis of whether traditional complexity-quality
168 relationships established for human-written code generalize to AI-generated code.

169 We extract metrics at the file level to ensure consistent comparison across all languages
170 and project structures. Metrics requiring cross-file dependencies or class hierarchy context
171 are excluded, as they do not exhibit meaningful variation in file-level analysis. All metric
172 extraction configurations and complete results are available in our replication package [1].

173 **5 Results**

174 **5.1 RQ1: TODO Question Summary**

175 TODO: Begin with a direct answer to RQ1. Present evidence before interpretation. Reference
176 tables and figures naturally. Discuss distributions, variability, and language-specific behavior
177 where relevant.

178 **5.2 RQ2: TODO Question Summary**

179 TODO: Begin with a direct answer to RQ2.

180 **5.3 RQ3: TODO Question Summary**

181 TODO: Begin with a direct answer to RQ3.

182 **6 Discussion**

183 **6.1 Summary of Findings**

184 TODO: Synthesize findings across all research questions. Connect results back to the empirical
185 software engineering literature.

186 **6.2 Implications for Practitioners**

187 TODO: Explain what the findings mean for software developers, teams, or tool builders.

188 **6.3 Implications for Researchers**

189 TODO: Identify open questions and directions for future empirical work.

190 **6.4 Threats to Validity**

191 **6.4.0.1 Construct Validity.**

192 TODO: Discuss whether the metrics and tools accurately capture the constructs of interest.

193 **6.4.0.2 Internal Validity.**

194 TODO: Discuss confounds and alternative explanations for the observed differences.

195 **6.4.0.3 External Validity.**

196 TODO: Discuss the generalizability of findings across languages, domains, and repository
197 types. Address dataset representativeness and language imbalance.

198 **6.4.0.4 Tooling Limitations.**

199 TODO: Discuss false positives, false negatives, and known limitations of the static analysis
200 or metric extraction tools used.

201 **References**

- 202 **1** Anonymous Authors. Replication package static analysis tool effectiveness for ai-generated
203 code. <https://github.com/anonymous/msr2026-replication>, 2026. Accessed 2026-04-14.
- 204 **2** CodeParrot. GitHub code clean dataset. [https://huggingface.co/datasets/codeparrot/](https://huggingface.co/datasets/codeparrot/github-code-clean)
205 [github-code-clean](https://huggingface.co/datasets/codeparrot/github-code-clean), 2022. Accessed 2025-05-01.
- 206 **3** CodeRabbit. State of AI vs human code generation report 2025. Technical report, CodeRabbit,
207 December 2025.
- 208 **4** B. Curtis, S. B. Sheppard, P. Milliman, M. Borst, and T. Love. Measuring the psychological
209 complexity of software maintenance tasks with the halstead and mccabe metrics. *IEEE*
210 *Transactions on Software Engineering*, SE-5(2):96–104, 1979.
- 211 **5** Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage:
212 A multi-lingual benchmark for detecting AI-generated and paraphrased source code from
213 production-level LLMs, 2025. URL: <https://arxiv.org/abs/2506.11059>, arXiv:2506.
214 11059.
- 215 **6** Dominik Honnef. staticcheck: The advanced go linter. <https://staticcheck.dev/>, 2024.
216 Accessed 2025-05-01.
- 217 **7** HTMLHint Contributors. HTMLHint: Static code analysis tool for HTML. [https://htmlhint.](https://htmlhint.com/)
218 [com/](https://htmlhint.com/), 2024. Accessed 2025-05-01.

- 219 8 Raphael Khoury, Anderson R. Avila, Jacob Brunelle, and Bhaskar Mitra Camara. On the secur-
220 ity vulnerabilities of text-to-SQL models. In *Proceedings of the 2023 IEEE/ACM International*
221 *Conference on Software Engineering*, 2023. Study cited in: [https://www.scworld.com/news/ai-](https://www.scworld.com/news/ai-will-create-a-tidal-wave-of-buggy-vulnerable-software)
222 [will-create-a-tidal-wave-of-buggy-vulnerable-software](https://www.scworld.com/news/ai-will-create-a-tidal-wave-of-buggy-vulnerable-software).
- 223 9 Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software
224 engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering, 2025.
225 AIDev-pop dataset Version 3 (August 2025 snapshot). URL: [https://arxiv.org/abs/2507.](https://arxiv.org/abs/2507.15003)
226 [15003](https://arxiv.org/abs/2507.15003), [arXiv:2507.15003](https://arxiv.org/abs/2507.15003).
- 227 10 Z. Liu, S. Wang, T. Chen, et al. Demystifying the code quality of ai-generated code: An
228 empirical study. In *Proceedings of the 38th IEEE/ACM International Conference on Automated*
229 *Software Engineering (ASE)*. IEEE, 2023.
- 230 11 Logilab. Pylint. <https://pylint.org/>, 2024. Accessed 2024-12-01.
- 231 12 Henry B. Mann. Nonparametric tests against trend. *Econometrica*, 13(3):245–259, 1945.
- 232 13 Daniel Marjamäki. Cppcheck: A tool for static c/c++ code analysis. [http://cppcheck.](http://cppcheck.sourceforge.net/)
233 [sourceforge.net/](http://cppcheck.sourceforge.net/), 2024. Accessed 2024-12-01.
- 234 14 Microsoft. Roslyn: The .net compiler platform. <https://github.com/dotnet/roslyn>, 2024.
235 Accessed 2024-12-01.
- 236 15 Ondrej Mirtes. PHPStan: PHP static analysis tool. <https://phpstan.org/>, 2024. Accessed
237 2025-05-01.
- 238 16 PMD. Pmd source code analyzer. <https://pmd.github.io/>, 2024. Accessed 2024-12-01.
- 239 17 Yujia Rao, Sridhar Chimalakonda, and Chakkrit Tantithamthavorn. Security weaknesses of
240 Copilot generated code in GitHub. *arXiv preprint arXiv:2310.02059*, October 2023. URL:
241 <https://arxiv.org/abs/2310.02059>, [arXiv:2310.02059](https://arxiv.org/abs/2310.02059).
- 242 18 RuboCop Contributors. RuboCop: A ruby static code analyzer and formatter. [https:](https://rubocop.org/)
243 [//rubocop.org/](https://rubocop.org/), 2024. Accessed 2025-05-01.
- 244 19 SciTools. Scitools understand: Static analysis tool. <https://scitools.com/>, 2024. Accessed
245 2024-12-01.
- 246 20 H. M. Sneed. Understanding software through numbers: A metric based approach to program
247 comprehension. *Journal of Software Maintenance: Research and Practice*, 7(6):405–419, 1995.
- 248 21 Nicholas C. Zakas. Eslint: Pluggable javascript linter. <https://eslint.org/>, 2024. Accessed
249 2024-12-01.
- 250 22 H. Zuse. Criteria for program comprehension derived from software complexity metrics. In
251 *1993 IEEE Second Workshop on Program Comprehension*, pages 8–16. IEEE, 1993.